DeepL Proに登録すると、より大きなサイズの文書ファイルを翻訳できます。
詳しくは、www.DeepL.com/pro をご覧ください。

モデル

プレハブの組織

小道具

モデルは以下のフォルダに：整理されている

家具：このフォルダのプレハブは通常静的で、剛体のないコライダだけを持って。

ギア道具と武器が。これらはキネマティック剛体。付きさまざまな素材タイプがあり、接頭辞が本来の解釈を示している：

- **P：**プリミティブ（木、石、骨）
- **B**または**Br：**青銅
- **I：**鉄
- **St:** スチール
- **Bs：**ブルー・スティール（より進化したファンタジー・スティール）
- **E：**エルヴン製（インゴットとして「マイトライト」と呼ばれる素材）
- **A：**アダマンタイン

ここでいう素材とは、日常的な意味でのあり、素材で素材で"ある "あって、レンダリングの。ためのユニティ素材ではないもちろん、ほとんどが色のバリエーションなので、これらの意味をどう解釈するかは自由です。

アイテム：これらは通常、動き回る可能性のある小さなものなので、キネマティック剛体が付属しています。もちろん、非キネマティックにすることもできますが、高価なサードパーティの物理ソリューション（Havokなど）を使用している場合を除き、そうすることは最良の結果をもたらさないかもしれません。これは、Unityのデフォルト物理の制限によるもので、非静的最適化のために組み込まれたメッシュコライダーがです。

アイテム・カテゴリーには、食品、ボトル／ボトル入り液体、本、食器／調理器具、楽器などがある。

自然：ここには岩のようなものが。ある

罠：罠と罠の部品はここにある。

Treasure：主にプレイヤーが戦利品として見つけるための財宝を目的としたアイテムのこと。絵画もここに含まれる。金や銀のカップや燭台は。素晴らしい戦利品になるが、同じようなアイテムをまとめておくために、価値の低いものと同じカテゴリーに分類されている

置く。インゴットも貴金属が多いのでここにあり、同じ種類のアイテムを同じ場所に保管するために、ここにある。価値の低いインゴットも

建築

建築部品は、ダンジョンの構造を簡単に組み立てることができるように設計されています。3m×3m、つまり10×10フィート（10フィートは3mより約1.3/4インチ、5cm小さい。テーブルトップゲームを模倣したこともあるし、一人称視点のゲームに理想的な結果だったからでもある。

これらは建築部品の：カテゴリーあるで

アーチ：主にドア用だが、その他にもいくつか。ポストやまぐさのドア枠もここにある。

天井ヴォールトアーチ型やドーム型の天井を作るためのモデル。これらは屋根ではなくおり、外からはあまりよく見えないかもしれません（どちらかというとダンジョン内部向け）。天井としてデザインされて

出入り口：これは完全な出入り口と、出入り口の。一部として穴が開いている壁の両方を含む

ドア：このカテゴリーには、未加工の（静的な）ドアとスクリプトで開けることのできるドア、物が含まれるそして出入り口の。開閉可能なドアの完全な合成城門や舷門もここにあります。

フェンス：これには）が含まれますモジュラーフェンス、ゲート、フェンスの、一部短い壁、ケージの壁（ケージのドアを含む。*シンプル・モジュラー・フェンスの*全内容はこちらをご覧ください。

フラット：すべての床と天井がここにある。これらのプレハブのほとんどは、上の階の天井としても床としても。使えるように設計されている穴のあいたフラットもあります。

柱柱はここにあります。これには、柱用のキャップ／柱頭、メインの柱ピース、コンポジット含まれます。完全な柱を、、形成する単一のメッシュとしてモデル化された完全な柱および柱として機能する大きな木柱がいくつかの*Simple Modular Pillars*のほとんどはここにありますが、一部はTrimの下にもあります（端の柱）。

プラットフォーム：これには含まれる2種類のプラットフォームが：

- ブロック：他のプレハブの下に敷くシンプルなブロック
- デイーズ玉座や祭壇のための段差のある高台

屋根家屋やコテージ（または城）になら勾配のある屋根が必要、ここにある。平らな屋根の場合は、Flatsをご利用ください。

階段：階段（3mまたは10フィートまで）、階段用手すり、より短く急でない階段（1.5mまたは5フィートまで）を含む。

トラップドアトラップドア／ハッチ（静的／装飾的、開閉可能）。

トリム： このカテゴリーには、壁を装飾する（あるいはヘアライン・クラックを隠す）端の柱や、装飾するモールディング含ま壁の上部や下部を、ダンジョンの壁に荒れ果てた感じを。与えるルース・レンガなどがれる

壁： 城や要塞の内壁と外壁の両方が含まれる。内壁には、薄いものと厚いもの、全長（3m）と半長（1.5m）、1つの材料と2つの材料、さらに斜めのバリエーションがある。半分の長さは、偶数の壁セグメントを使用する必要がある場合に、ドアを壁の。中央に配置できるようにするために追加された2つの材質の壁は、隣接する部屋に異なる材質の壁を持たせるためです。城壁と一緒に、欄干、城壁、門のための穴のある壁、フェイク/シェルタワーなど、いくつかの関連した機能も含まれています。(ただし、城門と舷門はドアの下の食べ物です)。

窓： 窓ガラス、窓シール、窓用の穴のあいた壁、なる合成物が含まれます完全な窓のある壁から。追加の窓ガラスも含まれており、追加したり入れ替えたりすることができます。

フェンスについて

モデルは基本的なモジュール式で、シーンに配置するためのものだ。エディターでの作り方を知っていれば、いるはずですこれらの。使い方を知って、プレハブを配置できるサードパーティ製ツールでも動作するはずです。

いくつかの命名規則がある：

- **生の**ゲートやドアは動かないもので、装飾や開閉できないバリアとして。これらは静止画として表示されます。使用することを目的としています

- **開閉可能な**ゲートやドアは、その名の通り、ゲームオブジェクトのれ、起動すると開くスクリプトが付属しています。ネストした階層として構築さ

- スタンドアローンを意味する**SAという**接尾辞は、両側に意味し、支柱があるフェンスセクションをダンジョン内の通路を塞いだり、「トラッド」セクションのループを閉じるなど、単独で。設置するのに適している

- **Tradと**付くセクションは、伝統的な方法でフェンスを作るためのもので、ポストのない方の端を次の端のポストに差し込む。そのため、片方の端にのみ支柱があり、もう片方の端は支柱を差し込むために少し長くなっている。

- **W**（の意*"whole "*）のラベルが貼られたセクションは、壁の。ように隣り合って "スナップ "するために、両端にハーフポスト（分割されたものちょうど半分に）を持っていますこれらは元々、特定のツールアセットで、残念ながら壊れた廃品になってしまいましたが、この方法で建設することを好むなら、通常のアセットとしてまだ使用できます。使用するためのもので

材料

このパックに含まれるほとんどのプレハブは単一の素材を使用しており、一部（特に宝石）はその素材の特別なバリエーションを使用している。このマテリアルは、美的な魅力とスタイル的な配慮のためにデザインされたカラーパレットをベースにしており、より明るい、特別な用途ために、より明るく、より彩度の高い色もありあるいは別の色が必要とされる（熱い石炭、宝石の石など）の、3つのテクスチャを使用することで、これらのベースカラーのメタリックとエミッシブのバリエーションを可能にしている。

また、ためにパレットにマッピングされた色が効果的でない。特殊なケースの、タイル状のテクスチャとマテリアルがいくつか含まれています

粒子と材料

炎、爆発、噴水など、いくつかのパーティクルシステムが含まれている。これらを使うのは難しくないし、好きなように改造することもできる。

スクリプト

様々なスクリプトが2つの主要パッケージに含まれています。1つ目はKFUnityUtilsで、ユーティリティとプリメイドシステムのです。これは主にドアを開けるスクリプトのために含まれており、開けるために再利用されていますファミリーチェストや引き出しを。もうひとつはシミュレーション・カメラで、これは主にデモでフライ・カムを作成する。ために含まれていますこれらのスクリプトはすべて（そしてスクリプトのみ）、このパックの一部として標準のUnityライセンスとMITライセンスの両方で利用可能です（どちらか好きな方を選択してください）。

KFUnityUtils

https://github.com/BlackJar72/KFUnityUtils

ドア開閉スクリプト

ドアを。開けるための、小さくてシンプルなスクリプトがいくつか含まれている

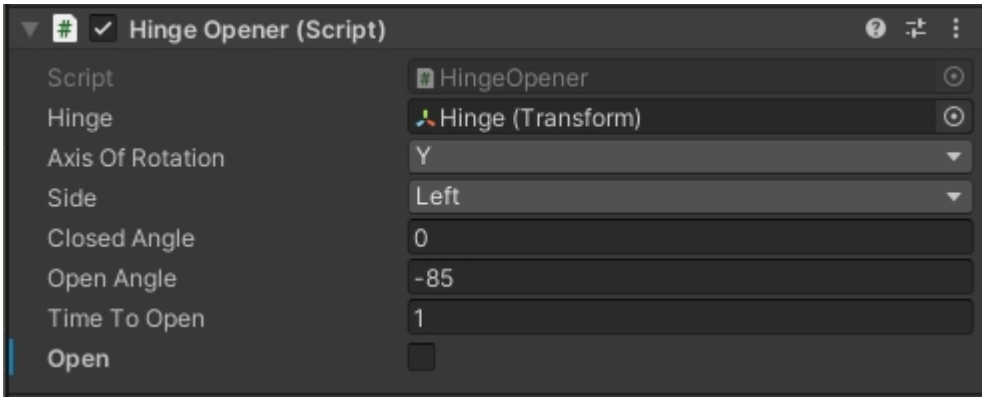
- ***IDoorOpener.***csは、ドアを開けるためのコア・メソッドを定義する基本インターフェースである。プログラマーが、ドアを動かすためのより多くの方法でシステムを拡張したい場合にのみ。役立つ
- ***SimpleOpener.***csはMonoBehaviourにIDoorOpenerインターフェイスを適用する抽象的な基本クラスで、インターフェイスに割り当てられたインスペクタに表示されないので必要です。
- ***HingeOpener.***csは、ピボット（ヒンジ）を中心にスイングしてドアやゲートなどを開けるスクリプトです。
- ***MovingOpener.***csは、ゲームオブジェクトを開閉位置の間で移動／並進させてドアを開けるスクリプト。です
- ***MultiOpener.***csは一度に複数のSimpleOpenersを開くスクリプト。ですこれは二重扉などにです。便利
- ***StagedOpener.***csは、ドア（または他のオブジェクト）をいくつかの移動させるスクリプトです。段階を経てこれは元々、秘密のドアのようなもののために意図されたもので、まず後ろに動いてから、下へスライドしたり、横にずれたりする。
-

すべてのスクリプトは、***Activate()***メソッドを呼び出すことでトリガーされるが、ドアを一方向にだけ動かしたい場合は、呼び出すこともできる。***Open()***メソッドとメソッドを***Close()***

スクリプトの使用

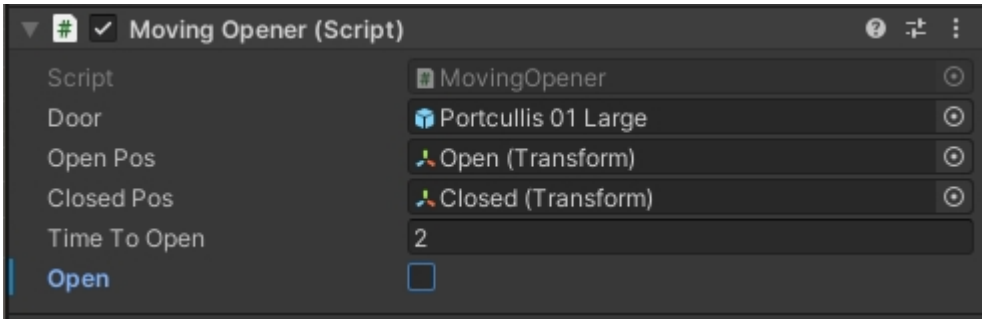
スクリプトを使用するには、開いている。プレハブにゲームオブジェクトの階層が必要です

ヒンジでスイングしたり、回転して開くドアを作るには、ヒンジトランスフォーム（通常は空）が必要です。ヒンジトランスフォームは、通常必要が、ドアの端か、端から少し離れた位置で、モデルあります。内の目に見えるヒンジと一直線上にあるドアは、ヒンジの子で。スクリプトは、ヒンジの上に置くことも、ゲームオブジェクトや空のオブジェクトの上に置くこともできます。なければなりません



スクリプト内の値は、ほぼ自明であろう：Hingeはヒンジの変形です。***Axis Of*** Rotationは回転軸で、ドアや門の場合は通常Yですが、チェストの場合はXかもしれません。***Side（側面）とは***、正面から見て二重扉である場合（あるいは二重扉であった場合）、指します。事実上、ドアに対してヒンジがある側を指します。、ドアがドアがある側を***クローズ・アングルは***、ドアが閉まったときの角度で、***オープン・***アングルは完全に開いたときの角度です。***Time To Open（開くまでの時間）***とは、単純にドアが開いている状態から閉じている状態、またはその逆の状態になるまでの時間を表します。最後に、Openはドアが現在開いているかどうかを表し、インスペクターからドアの開始位置を設定するために使用できます。

城の入り口に、ムービング・オープナー・スクリプトを使用します。ある昇降可能な舷門など、動いて開くドアや門を作るには



動くゲームオブジェクトであるドアやゲートは、2つのトランスフォーム（通常は空）を持つコンテナに入れ、ゲームオブジェクトが開いたときと閉じたときにあるべき場所に配置する必要があります。コンテナは、空でもいいし、オブジェクトの一部でも、くっついたものでもいい。必要が変数はインスペクタで設定するある、これらの変数に加え、***Time 時間To Open（開くまでの***）と***Open（開く***）です。

MultiOpenerスクリプトは、他の単純なオープナー（ヒンジオープナー、含む）のリストを作成しムービングオープナー、ステージドオープナー、あるいは他のマルチオープナーを、そのリストがアクティブにアクティブになるようになったときにします。すべてが

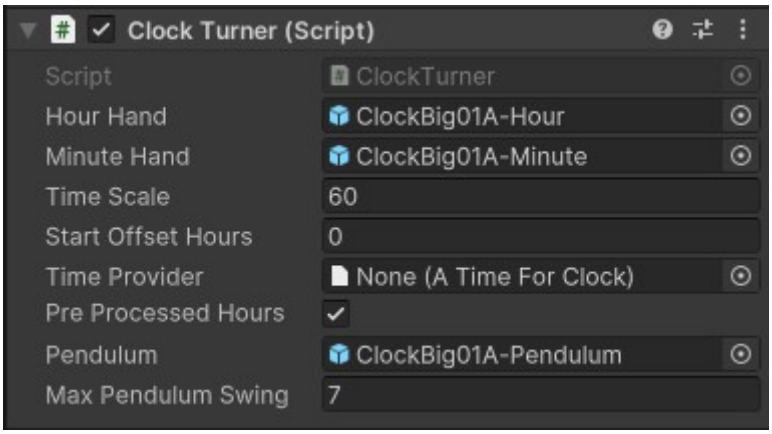
別のスクリプトからActivate、Open、Closeを呼び出すか、インスペクタに配線されたunityイベントを呼び出すだけで、動くようになるはずだ。もちろん、ドアやゲート以外にも使える。チェストの蓋や、刃を振り出したり何かでジャブしたりするトラップも、このアニメーションさせることができる。スクリプトで

ステージド・オープナーは、一連のトランスフォームを提供するだけで、ポジションを移動し、そのポジションに到達するまでの時間のリストを提供する。

その他のクラス（利用直下）

クロック・ターナー.cs

このクラスは、時計をアニメーション化し、時を。刻むために使われる



Hour （Hand時針）と **Minute Hand（分針）**は単に時計の針（ゲームオブジェクト）で、Z軸を中心に回転して時を刻みます。それぞれのゲームオブジェクトを用意する必要があります（用意しないとクラッシュします）。もし、両方の針を用意したくない場合は、空の針を用意する必要があります。

Time タイムScale（スケール）とは、時間のことです。デフォルトの60は1日を24分とし、ゲームワールドの世界では現実時間の1分が1時間となります。同様に、24は1時間を表します。Mojangを真似るなら72で1日、20分Bethesdaを真似るなら20で1日72分、1なら完全にリアルタイムになります。

Start Offset 時刻は、時計を開始する時刻（試合。開始時の時刻）を示す

タイムプロバイダーは*ClockTurner.ATimeForClock*から派生したスクリプト可能なオブジェクトです。これは、ワールドタイムシステムがある場合、それにクロックをフックするためのものです。これが "none "のままだと、クロックはUnityの基づきます。*Time.time*.Timeに

Pre Processinged Hoursは、示しますTime Provideからの時間がすでにその日の。時間を提供しているかどうかをもしそうであれば、Time ScaleとStart Offset Hoursは何もしません。これがfalseの場合、Time Providerがゲーム開始からの生の秒を提供しているとみなされ、Time ScaleとOffsetが適用されます。

Pendulum は振り子を表すオプションのゲームオブジェクトです。時計に振り子がない場合は "none" にしてください。

Max Pendulum Swing は振り子が何度揺れるかを示す。もし振り子がなければ、これは何もしない。

KFMath.cs

様々な用途のための数学関数。

`public static int ModRight(int a, int b)`：これはモジュロ(剰余)を生成しますが、最も素朴に期待されるように、そして多くの状況でより有用であるように、小さい数から始まる負の数を生成します。

`public static float Asymptote(float n, float start, float rate) public static double Asymptote(double n, double start, double rate)`

nがstartより小さいか等しい場合、これはnを返す。nがstartより小さいか等しい場合、返される***nが。nがstartより***大きい場合、***nが***無限大で***start*+ *rate***に**到達するような、***start*+ *rate***に**近づく値が返され、***nが***2倍になるたびに***start*+ *rate***との**差が半分になる。

`public static long GetLongSeed(this string str) public static ulong GetULongSeed(this string str) public static int GetIntSeed(this string str)`

これらは文字列を数値に。変換する文字列が指定した型の、その数値を返します。数値を表す表現で有効なあればそうでない場合は、文字列のハッシュを返します。これは主に乱数ジェネレータの種を生成するために作成されました。

シャフラー.cs

このクラスは **`IList<T>` を**実装したシャッフルリストに追加される便利なメソッドを含んでいます。

`public static void Shuffle<T>(this IList<T> list)` ： UnityのRandomクラスを使って。リストをシャッフルします

`public static void Shuffle<T>(this IList<T> list, Xorshift random)` ： このライブラリの Xorshift クラスを使ってリストを。シャッフルします

SpatialHash.cs

このクラスは空間内の位置に基づいて乱数を生成するためのものである。メソッドは典型的な乱数生成器と似ているが、範囲形がなく、すべて3つまたは座標を取る。技術的には、整数座標（x、y、z持つ4次元空間の点に基づいている。、t）をx、z、オプションのyは位置を決定するためのもので、t（時間）は各点が完全な数列を持つことを。可能にするまた、1つの変数以外をすべて定数に保つことで、乱数列のどの番号でも、必要なときに必要なだけ、乱数列を生成することなく見つけることができる。

これはもともと手続き的ために作られたものだが、ハッシュを割り当てるのにも役立って生成の*Dictionary<K,V>*クラスのようなハッシュ・テーブルで使うために、位置に基づいている。

TransformData.cs

実際のトランスフォーム以外のトランスフォームデータを格納するための構造体。

Xorshift.cs

代替乱数ジェネレータです。これはもともと、UnityのRandomクラスの使用によって影響を受けない、割り当て可能なシードから乱数のプールを作成するために作成されました。これは元々プロシージャルなレベル生成の、ためのものでプレイヤーが提供したシードをし、ゲームプレイのために生成された乱数の使用による影響を受けないようにするためのものでした。使用してレベルを生成

シミュレーションカメラ

https://github.com/BlackJar72/Simulation-Game-Cameras

シティビルダー、ライフシミュレーター、ターン制/リアルタイムストラテジーなどのシミュレーションゲームやストラテジーゲームで使用されるカメラコントロールシステム。

これらのスクリプトは、主にデモシーンの。フライカムを提供するために含まれているスクリプトはデモに付属していますが、ここでは特に説明しません。

カメラの種類を入れ替える機能や、結果を処理するために。イベントを作成するシステムが組み込まれている

カメラの種類

クラシック・コントロール（`ClassicControl.cs`）： シティービルダーの典型的なコントロールで、画面の端を押して動かす。カメラをマウスホイールはズームインとズームアウトを行い、マウスの中クリックはズームレベルをデフォルトに戻す。WASDはカメラを回転させ、[shift]を押し続けるとWASDに基づいた移動に変わる。垂直飛行が有効な場合、QとEでカメラが上下する。

離散Yによるクラシック・コントロール（`ClassicDiscreteYControl.cs`）： クラシックコントロールに似ていますが、います。**Q**（下）と**E**（上）で移動する離散Yレベルを備えてこれは、家やビルのフロア間を移動するようなライフシミュレーションゲームで。使用するために設計されました

ファーストパーソンコントロール（`FirstPersonControl.cs`）： **スペース]**で上に飛び、**[シフト]**で下に飛びます。これは、ボクセルベースのビルディングゲームのクリエイティブモードや観戦モードで見られるものに似て。いますルックディレクションは水平面（x、z）内の移動にしか影響しないので、高さを変えずに上や下を見ることができる。おそらく、この方法で都市を建設したい人もいるだろう。

離散Yを持つ一人称コントロール（`FirstPersonDiscreteYControl.cs`）： 同様の一人称コントロールですが、離散Yを持つクラシックコントロールに似た離散Yレベルを持ちます。

フリーカメラコントロール（`FreeCamControl.cs`）： 一人称視点のフライカム。**スペース]**と**[シフト]**で「上」と「下」に飛ぶこともできます。
どうやら、なぜかこの手のカメラが好きな人がいるようだ。

すべてのカメラは左クリックと右クリックを同じように処理し、何かがクリックさするとそれをマウスの左ボタンボタンでまたは右れたり、ホールドされたり、検出して、他の場所で使用できる。イベントを送信します

その他のクラス

ACameraControl.cs：すべてのカメラコントロールの。抽象ベース共通機能をし、他のカメラコントロールを作成してモードスイッチャーで使用できるようにする。実装

EventConverter.cs：カメラコントローラからのイベントを処理し、適切なUnityイベントに。変換します
ModeSwitcher.cs：ゲーム中のカメラ制御を切り替える機能の基礎となる。モードスイッチャーでカメラをセットアップするには、このスクリプトがアタッチさ。希望するすべてのカメラコントローラー（上記のリストから）を順番に追加し、それぞれを非アクティブに設定する必要があります。れた空の子である必要がありますカメラはPlayer Eyeフィールドに追加しください、各コントローラの、その他の変数は、それぞれ。必要に応じて設定してデモシーンにはゲームオブジェクトの "Camera Holder "の例があります。

デフォルトでは、**F**キーでカメラコントローラーを。切り替えます